

实验四：MongoDB 查询文档

【实验目的】1. 掌握 MongoDB 查询文档

【实验原理】

1. 使用 find() 方法进行文档基本查询

语法格式如下：

`db.collection.find(query, projection)`

参数说明：

query：可选，使用查询操作符指定查询条件

projection：可选，使用投影操作符指定返回的键。查询时返回文档中所有键值，只需省略该参数即可（默认省略）。

2. 文档查询条件的使用

操作	格式	范例	RDBMS 中的类似语句
等于	{<key>:<value>}	db.col.find({"by":"x"}).pretty()	where by = 'x'
小于	{<key>:{<\$lt:<value>}}	db.col.find({"likes":{\$lt:50}}).pretty()	where likes < 50
小于或等于	{<key>:{<\$lte:<value>}}	db.col.find({"likes":{\$lte:50}}).pretty()	where likes <= 50
大于	{<key>:{<\$gt:<value>}}	db.col.find({"likes":{\$gt:50}}).pretty()	where likes > 50
大于或等于	{<key>:{<\$gte:<value>}}	db.col.find({"likes":{\$gte:50}}).pretty()	where likes >= 50
不等于	{<key>:{<\$ne:<value>}}	db.col.find({"likes":{\$ne:50}}).pretty()	where likes != 50

3. 特定类型查询

针对特定类型的文档进行查询，如查询键为 NULL 的空文档

4. 聚合查询

MongoDB 中聚合(aggregate)主要用于处理数据(诸如统计平均值,求和等), 并返回计算后的数据结果。db.collection.aggregate()是基于数据处理的聚合管道, 每个文档通过一个由多个阶段(stage)组成的管道, 可以对每个阶段的管道进行分组、过滤等功能, 然后经过一系列的处理, 输出相应的结果。

表达式	描述	实例
\$sum	计算总和。	db.mycol.aggregate([{\$group: {_id: "\$by_user", num_tutorial: {\$sum: "\$likes"}}}])
\$avg	计算平均值	db.mycol.aggregate([{\$group: {_id: "\$by_user", num_tutorial: {\$avg: "\$likes"}}}])
\$min	获取集合中所有文档对应值得最小值。	db.mycol.aggregate([{\$group: {_id: "\$by_user", num_tutorial: {\$min: "\$likes"}}}])
\$max	获取集合中所有文档对应值得最大值。	db.mycol.aggregate([{\$group: {_id: "\$by_user", num_tutorial: {\$max: "\$likes"}}}])
\$push	在结果文档中插入值到一个数组中。	db.mycol.aggregate([{\$group: {_id: "\$by_user", url: {\$push: "\$url"}}}])
\$addToSet	在结果文档中插入值到一个数组中, 但不创建副本。	db.mycol.aggregate([{\$group: {_id: "\$by_user", url: {\$addToSet: "\$url"}}}])
\$first	根据资源文档的排序获取第一个文档数据。	db.mycol.aggregate([{\$group: {_id: "\$by_user", first_url: {\$first: "\$url"}}}])

\$last	根据资源文档的排序获取最后一个文档数据	db.mycol.aggregate([{\$group : {_id : "\$by_user", last_url : {\$last : "\$url"}}}])
【实验环境】		
OS: Ubuntu16.04/CentOS7 or Windows 7/10		
MongoDB: v3.6 from l9000 or v4.2.19		
【实验步骤】		
1. 使用 test 数据库，并新建 items 集合，保存订单相关信息。 <pre>db.createCollection("items")</pre> 2. 插入文档数据： 每个文档对应订单中某个商品相关信息，包括： pnumber: 商品编号 quantity: 商品数量 price: 商品单价 插入如下商品信息： <pre>db.items.insert([{"quantity":2,price:5.0,pnumber:"p003"}, {"quantity":2,price:8.0,pnumber:"p002"}, {"quantity":1,price:4.0,pnumber:"p002"}, {"quantity":2,price:4.0,pnumber:"p001"}, {"quantity":4,price:10.0,pnumber:"p003"}, {"quantity":10,price:20.0,pnumber:"p001"}, {"quantity":10,price:20.0,pnumber:"p003"}, {"quantity":5,price:10.0,pnumber:"p002"}])</pre> 结果：		

```
> db.items.insert( [{"quantity" : 2,"price" : 5.0,"pnumber" : "p003"}, {"quantity" : 2,"price" : 8.0,"pnumber" : "p002"}, {"quantity" : 1,"price" : 4.0,"pnumber" : "p002" }, {"quantity" : 2, "price" : 4.0, "pnumber" : "p001" }, {"quantity" : 4, "price" : 10.0, "pnumber" : "p003" }, {"quantity" : 10,"price" : 20.0,"pnumber" : "p001" }, {"quantity" : 10, "price" : 20.0,"pnumber" : "p003" }, {"quantity" : 5,"price" : 10.0,"pnumber" : "p002" }])
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 8,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
```

3. 查询插入结果:

db.items.find().pretty() : pretty() 方法以格式化的方式来显示所有文档

```
> db.items.find().pretty()
{
  "_id" : ObjectId("5b67f7a29fd52eadb83c3ea4")
  "quantity" : 2,
  "price" : 5,
  "pnumber" : "p003"
}
{
  "_id" : ObjectId("5b67f7a29fd52eadb83c3ea5")
  "quantity" : 2,
  "price" : 8,
  "pnumber" : "p002"
}
{
  "_id" : ObjectId("5b67f7a29fd52eadb83c3ea6")
  "quantity" : 1,
  "price" : 4,
  "pnumber" : "p002"
}
{
  "_id" : ObjectId("5b67f7a29fd52eadb83c3ea7")
  "quantity" : 2,
  "price" : 4,
```

4. 统计items共有多少个文档数据

db.items.count()

```
> db.items.count()
8
```

5. 查询价格大于 5 的商品数据

大于使用 gt 操作符, 另外操作符前面要带上\$符号

```
db.items.find({price:{$gt:5}})
> db.items.find({price:{$gt:5}})
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3ea5"), "quantity" : 2, "price" : 8, "number" : "p002" }
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3ea8"), "quantity" : 4, "price" : 10, "number" : "p003" }
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3ea9"), "quantity" : 10, "price" : 20, "number" : "p001" }
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3eaa"), "quantity" : 10, "price" : 20, "number" : "p003" }
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3eab"), "quantity" : 5, "price" : 10, "number" : "p002" }
```

6. 多条件查询

例：查询quantity为10且价格大于等于5的商品数据

```
db.items.find({quantity:10,price:{$gte:5}})
> db.items.find({quantity:10,price:{$gte:5}})
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3ea9"), "quantity" : 10, "price" : 20, "number" : "p001" }
{ "_id" : ObjectId("5b67f7a29fd52eadb83c3eaa"), "quantity" : 10, "price" : 20, "number" : "p003" }
```

7. 使用or来进行条件查询，格式如下：

```
db.col.find(
  {
    $or:
    [
      {key1: value1}, {key2:value2}
    ]
  }
)
```

例：查询quantity为10或价格大于等于5的商品数据

```
db.items.find({$or:[{quantity:10},{price:{$gte:5}}]})
> db.items.find({$or:[{quantity:10},{price:{$gte:5}}]})
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8eb"), "quantity" : 2, "price" : 5, "number" : "p003" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8ec"), "quantity" : 2, "price" : 8, "number" : "p002" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8ef"), "quantity" : 4, "price" : 10, "number" : "p003" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8f0"), "quantity" : 10, "price" : 20, "number" : "p001" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8f1"), "quantity" : 10, "price" : 20, "number" : "p003" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8f2"), "quantity" : 5, "price" : 10, "number" : "p002" }
```

8. AND 和 OR 联合使用

例：查询pnumber为“p003”且quantity为10或价格大于等于5的商品数据

```
db.items.find({pnumber:"p003",$or:[{quantity:10},{price:{$gte:5}}]})
> db.items.find({pnumber:"p003",$or:[{quantity:10},{price:{$gte:5}}]})
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8eb"), "quantity" : 2, "price" : 5, "number" : "p003" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8ef"), "quantity" : 4, "price" : 10, "number" : "p003" }
{ "_id" : ObjectId("5c16ff7522d68cf70c57a8f1"), "quantity" : 10, "price" : 20, "number" : "p003" }
```

9. 使用聚合aggregate

例：统计订单中所有商品的数量，即统计quantity的总和。

```
db.items.aggregate([{$group:{_id:null,total:{$sum: "$quantity"}}}])
> db.items.aggregate([{$group:{_id:null,total:{$sum: "$quantity"}}}])
{ "_id" : null, "total" : 36 }
```

例：通过产品类型来进行分组，然后在统计卖出的数量

```
db.items.aggregate([{$group:{_id:"$pnumber",total:{$sum: "$quantity"}}}])
```

```
> db.items.aggregate([{$group:{_id:"$pnumber",total:{$sum:"$quantity"}}}])
{ "_id" : "p001", "total" : 12 }
{ "_id" : "p002", "total" : 8 }
{ "_id" : "p003", "total" : 16 }
```

例：通过相同的产品类型来进行分组，然后查询相同产品类型卖出最多的订单详情。

```
db.items.aggregate([{$group:{_id:"$pnumber",max:{$max:"$quantity"}}}])
> db.items.aggregate([{$group:{_id:"$pnumber",max:{$max:"$quantity"}}}])
{ "_id" : "p001", "max" : 10 }
{ "_id" : "p002", "max" : 5 }
{ "_id" : "p003", "max" : 10 }
```

例：通过相同的产品类型来进行分组，然后查询每个订单详情相同产品类型卖出的平均价格

```
db.items.aggregate([{$group:{_id:"$pnumber",price:{$avg:"$price"}}}])
> db.items.aggregate([{$group:{_id:"$pnumber",price:{$avg:"$price"}}}])
{ "_id" : "p001", "price" : 12 }
{ "_id" : "p002", "price" : 7.333333333333333 }
{ "_id" : "p003", "price" : 11.666666666666666 }
```

10. 管道的使用

例：通过相同的产品类型来进行分组，统计各个产品数量，然后获取最大的数量。

```
db.items.aggregate([{$group:{_id:"$pnumber",total:{$sum:"$quantity"}}},{ $group:{_id:null,max:{$max:"$total"}}}])
> db.items.aggregate([{$group:{_id:"$pnumber",total:{$sum:"$quantity"}}},{ $group:{_id:null,max:{$max:"$total"}}}])
{ "_id" : null, "max" : 16 }
```